

Module 7: Object-Oriented Programming (Classes, Objects, Constructors, `this`, `static`, `final`, Encapsulation)

Java Laboratory – Scenario-Based Problem Statements

Module Overview

Object-Oriented Programming (OOP) is the foundation of modern software development. Almost every enterprise application—such as banking systems, hospital management systems, e-commerce platforms, ERP systems, and mobile applications—is built using object-oriented principles.

This module introduces students to the core concepts of OOP by designing and implementing **real-world business applications** using **Classes, Objects, Constructors, the `this` keyword, `static` members, `final` keyword, and Encapsulation**. The laboratory exercises encourage students to think like software engineers by modeling real-world entities as objects.

Total Practice Problems: 25

Difficulty Distribution

- **Beginner:** 8 Problems
 - **Intermediate:** 10 Problems
 - **Advanced:** 7 Problems
-

Learning Objectives

After completing this module, students will be able to:

- Design classes and objects representing real-world entities.
- Implement constructors for object initialization.
- Use the `this` keyword effectively.
- Understand and apply `static` members.
- Apply the `final` keyword appropriately.
- Implement encapsulation using private fields and public getter/setter methods.
- Develop modular object-oriented applications.

Concepts Covered

- Classes and Objects
- Instance Variables
- Constructors
- Constructor Overloading
- `this` Keyword
- `static` Variables
- `static` Methods
- `final` Variables
- `final` Methods
- Encapsulation
- Getters and Setters

Beginner Level Problems

Problem 1: Student Information Management

Scenario

A university wants to maintain student records using an object-oriented approach.

Problem Statement

Design a `Student` class with the following attributes:

- Student ID
- Name
- Branch
- Semester
- CGPA

Create methods to:

- Register a student
- Display student details
- Update CGPA

Use constructors to initialize objects.

Problem 2: Employee Information System

Scenario

A company wants to maintain employee records.

Problem Statement

Create an `Employee` class containing:

- Employee ID
- Name
- Department
- Designation
- Salary

Provide constructors for initialization and methods to display employee information.

Problem 3: Library Book Management

Scenario

A college library stores information about books.

Problem Statement

Design a `Book` class with attributes:

- Book ID
- Title
- Author
- Publisher
- Price

Implement constructors and methods to display book details.

Problem 4: Bank Customer Profile

Scenario

A bank maintains customer profiles.

Problem Statement

Create a `Customer` class with:

- Customer ID
- Name
- Mobile Number
- Email
- Address

Provide methods to update customer information and display the profile.

Problem 5: Vehicle Registration System

Scenario

A Regional Transport Office (RTO) registers vehicles.

Problem Statement

Design a `Vehicle` class containing:

- Registration Number
- Owner Name
- Vehicle Type
- Manufacturer
- Model

Create objects for multiple vehicles and display their information.

Problem 6: Hospital Patient Registration

Scenario

A hospital maintains patient records.

Problem Statement

Create a `Patient` class with:

- Patient ID
- Name
- Age
- Blood Group
- Disease

Use constructors to initialize patient information and display the records.

Problem 7: Mobile Phone Inventory

Scenario

A mobile store manages inventory.

Problem Statement

Create a `MobilePhone` class containing:

- Brand
- Model
- RAM
- Storage
- Price

Allow multiple mobile objects to be created and displayed.

Problem 8: Product Catalog

Scenario

An e-commerce company maintains product information.

Problem Statement

Create a `Product` class with:

- Product ID
- Product Name
- Category
- Price
- Stock Quantity

Provide methods to display product details.

Intermediate Level Problems

Problem 9: Banking Account Management

Scenario

A bank wants to implement customer accounts using OOP.

Problem Statement

Design a `BankAccount` class.

Include methods for:

- Deposit
- Withdraw
- Balance Inquiry

Use encapsulation to protect account details.

Problem 10: Hotel Room Reservation

Scenario

A hotel manages room bookings.

Problem Statement

Design a `Room` class with:

- Room Number
- Room Type

- Tariff
- Availability Status

Provide methods for booking and cancellation.

Problem 11: Shopping Cart System

Scenario

An online shopping portal allows customers to purchase products.

Problem Statement

Create classes for:

- Product
- Customer
- ShoppingCart

Allow customers to add products and calculate the total bill.

Problem 12: College Course Registration

Scenario

Students register for different academic courses.

Problem Statement

Create a `Course` class.

Allow:

- Course creation
 - Student enrollment
 - Display of registered courses
-

Problem 13: Constructor Overloading – Employee Management

Scenario

A company receives employee information in different formats.

Problem Statement

Implement constructor overloading to initialize employee objects using:

- Default constructor
 - Constructor with basic details
 - Constructor with complete details
-

Problem 14: Banking Application Using `this`

Scenario

A bank application receives parameters having the same names as instance variables.

Problem Statement

Design a `BankCustomer` class and use the `this` keyword to differentiate between local variables and instance variables during object initialization.

Problem 15: Company Information Using `static`

Scenario

Every employee in a company belongs to the same organization.

Problem Statement

Create an `Employee` class where:

- Company Name
- Company Address
- HR Contact

are declared as `static` members.

Display how these values are shared among all employee objects.

Problem 16: University Fee Structure

Scenario

A university has a fixed registration fee applicable to all students.

Problem Statement

Use a `static` variable to represent the registration fee.

Allow multiple student objects to calculate the total admission fee.

Problem 17: Product Price Protection

Scenario

A manufacturer assigns a fixed product serial number that should never change.

Problem Statement

Use the `final` keyword to protect the product serial number from modification.

Problem 18: Library Membership System

Scenario

A library wants to hide sensitive member information.

Problem Statement

Implement encapsulation by making all data members private.

Provide getter and setter methods with appropriate validation.

Advanced Level Problems

Problem 19: University Management System

Scenario

A university wants to manage students, faculty, and courses.

Problem Statement

Design separate classes for:

- Student
- Faculty
- Course
- Department

Use object composition where appropriate.

Implement constructors and encapsulation.

Problem 20: Hospital Information System

Scenario

A hospital stores patient records, doctor details, and appointments.

Problem Statement

Create classes for:

- Patient
- Doctor
- Appointment

Ensure proper object interaction while maintaining encapsulation.

Problem 21: Airline Reservation System

Scenario

An airline manages passengers and flight bookings.

Problem Statement

Create classes for:

- Passenger
- Flight
- Ticket

Use constructors for object initialization and static variables for airline information.

Problem 22: Retail Inventory Management

Scenario

A retail company maintains product inventory.

Problem Statement

Design classes for:

- Product
- Supplier
- Inventory

Use encapsulation to prevent invalid stock updates.

Problem 23: Smart Banking System

Scenario

A bank wants to securely manage customer accounts.

Problem Statement

Implement classes for:

- Customer
- Account
- Transaction

Protect sensitive information using encapsulation and use `static` members for bank-related information.

Problem 24: Online Food Delivery System

Scenario

A food delivery platform maintains restaurant and order information.

Problem Statement

Create classes for:

- Restaurant
- MenuItem
- Customer
- Order

Use constructors for initialization and encapsulation for data security.

Problem 25: College ERP System

Scenario

A college wants to digitize academic operations.

Problem Statement

Develop an application containing classes for:

- Student
- Faculty
- Department
- Course
- Examination

Use constructors, `this`, `static`, `final`, and encapsulation wherever appropriate.

Generate a consolidated academic report.

Challenge Problems

1. Develop a **Banking Management System** with multiple account types using classes and encapsulation.
 2. Design a **Hospital Management System** with patient registration, doctor allocation, and appointment scheduling.
 3. Build a **Library Automation System** with books, members, librarians, and issue/return functionality.
 4. Create an **Online Shopping Platform** featuring products, customers, carts, and orders.
 5. Develop a **Hotel Reservation System** with rooms, guests, bookings, and billing.
 6. Design a **College ERP System** that manages admissions, attendance, examinations, and fee records.
 7. Implement a **Vehicle Rental Management System** using classes for customers, vehicles, rentals, and invoices.
-

Real-World Applications

The concepts covered in this module are widely used in:

- Banking and Financial Systems
 - Hospital Management Systems
 - University ERP Solutions
 - Hotel Reservation Platforms
 - Airline Reservation Systems
 - Retail Inventory Systems
 - E-commerce Applications
 - Government Citizen Portals
 - Enterprise Resource Planning (ERP)
 - Customer Relationship Management (CRM)
-

Common Mistakes

Students should avoid:

- Making all variables public instead of using encapsulation.
 - Writing business logic directly in the `main()` method.
 - Forgetting to initialize objects using constructors.
 - Misusing the `this` keyword.
 - Declaring instance variables as `static` without justification.
 - Attempting to modify `final` variables after initialization.
 - Creating overly large classes with multiple responsibilities.
 - Omitting validation in setter methods.
-

Viva Questions

1. What is Object-Oriented Programming (OOP)?
 2. Differentiate between a class and an object.
 3. What is the purpose of a constructor? How does it differ from a method?
 4. Explain constructor overloading with an example.
 5. What is the purpose of the `this` keyword?
 6. Differentiate between instance variables and `static` variables.
 7. When should a `static` method be used?
 8. Explain the purpose of the `final` keyword for variables, methods, and classes.
 9. What is encapsulation, and how does it improve software quality?
 10. Why should fields generally be declared `private`?
-

Expected Learning Outcomes

After completing this module, students will be able to:

- Model real-world entities as Java classes and objects.
- Use constructors to initialize object state effectively.
- Apply the `this` keyword to improve code clarity.
- Share common information across objects using `static` members.
- Protect constant values using the `final` keyword.
- Implement encapsulation through private fields and controlled access methods.
- Design maintainable, reusable, and scalable object-oriented applications aligned with industry best practices.

Special Notes

- Encourage students to identify **real-world entities**, their **attributes**, and **behaviors** before writing any code.
- Reinforce industry-standard coding practices such as private data members, meaningful method names, constructor chaining where appropriate, and validation within setter methods.
- Discuss how these foundational OOP concepts prepare students for advanced topics such as inheritance, polymorphism, abstraction, design patterns, and enterprise frameworks like Spring Boot.